HomeWork1

Aswath Oruganti

May 25, 2026

1 Concept Questions

1. Equivalence of clustering assignment formulations

Show that the standard clustering assignment problem based on the squared Euclidean distance:

$$j^{(i)} = \arg\min_{j=1,\dots,k} \|x_i - c_j\|^2 \tag{1}$$

is mathematically equivalent to solving the simplified formulation:

$$j^{(i)} = \arg\min_{j=1,\dots,k} (c_j)^T \left(\frac{1}{2}c_j - x_i\right) \tag{2}$$

1.1 Analytical proof of equivalence

We can algebraically expand the objective function inside the arg min operator of the standard formulation to establish the equivalence between the above two formulations. The squared Euclidean distance (L_2 norm) between a data point $x_i \in \mathbb{R}^n$ and a cluster centroid $c_j \in \mathbb{R}^n$ can be expressed as the vector inner product of their difference with itself:

$$\|x_i - c_j\|^2 = (x_i - c_j)^T (x_i - c_j) \tag{3}$$

Expanding the vector transposition yields:

$$\begin{aligned} \|x_i - c_j\|^2 &= (x_i^T - c_j^T)(x_i - c_j) \\ &= x_i^T x_i - x_i^T c_j - c_j^T x_i + c_j^T c_j \end{aligned} \tag{4}$$

Because the inner product of two vectors is a scalar, the operation is commutative, meaning the order of multiplication does not matter ($x_i^T c_j = c_j^T x_i$). Combining these middle terms yields:

$$\|x_i - c_j\|^2 = x_i^T x_i - 2(c_j)^T x_i + (c_j)^T c_j \tag{5}$$

We now substitute this expanded trinomial back into the original optimization operator:

$$j^{(i)} = \arg\min_{j=1,\dots,k} \left(x_i^T x_i - 2(c_j)^T x_i + (c_j)^T c_j\right) \tag{6}$$

1.2 Dropping the Constant Shift Term

The optimization operator arg min evaluates candidate choices along the cluster index j. The first term, $x_i^T x_i$, depends exclusively on the specific data point x_i and remains completely constant across all choices of j.

Adding or subtracting a constant across all candidates preserves their relative ordering and does not change the location of the minimum. Therefore, we can safely drop the term $x_i^T x_i$ without altering the optimal index outcome:

$$j^{(i)} = \arg\min_{j=1,\dots,k} \left((c_j)^T c_j - 2(c_j)^T x_i \right) \tag{7}$$

1.3 Constant Positive Scaling & Factoring

Multiplying an objective function by a strictly positive scalar uniformly scales the resulting distances but preserves their monotonic ordering. We scale the objective function inside the operator by a positive factor of $\frac{1}{2}$:

$$j^{(i)} = \arg\min_{j=1,\dots,k} \left(\frac{1}{2}(c_j)^T c_j - (c_j)^T x_i \right) \tag{8}$$

Finally, by applying the distributive property of matrix multiplication, we factor out the shared row vector $(c_j)^T$ from both remaining terms:

$$j^{(i)} = \arg\min_{j=1,\dots,k} (c_j)^T \left(\frac{1}{2} c_j - x_i \right) \tag{9}$$

This exactly matches the target expression, completing the analytical proof.

1.4 Computational Result Summary

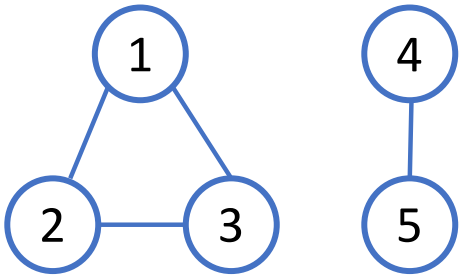
- **Mathematical Equivalence:** The proof demonstrates that minimizing geometric distance to a cluster center is mathematically identical to finding a penalized vector projection.
- **Algorithmic Limitation of Equation (1):** Direct calculation of $\|x_i - c_j\|^2$ requires element-wise subtraction loops or expensive memory broadcasting arrays across every data-point-to-centroid pair.
- **Vectorization Advantage of Equation (2):** The reformulated expression decouples the data terms from the purely centroid-dependent terms. This allows the cluster assignment operation for an entire dataset matrix $\mathbf{X}$ and centroid matrix $\mathbf{C}$ to be offloaded into a single, highly optimized **Level 3 BLAS Matrix Multiplication (GEMM)** routine:

  $$\mathbf{A} = \mathbf{C}^T \left(\frac{1}{2}\mathbf{C} - \mathbf{X} \right)$$

  This eliminates loop overhead and provides massive execution speedups in modern data science libraries.

2. Graph Laplacian and Spectral Clustering

Consider the following simple graph:



Write down the graph Laplacian matrix and find the eigenvectors associated with the zero eigenvalues. Explain how you find out the number of disconnected clusters in the graph and identify these disconnected clusters using these eigenvectors.

2.1 Spectral Graph Theory Analysis

To solve this spectral graph theory problem, we will break it down into three steps: constructing the matrices, finding the specific eigenvectors, and explaining the connection between zero eigenvalues and the graph's components.

2.2 Write Down the Graph Laplacian Matrix

The graph Laplacian matrix L is defined as $L = D - A$, where D is the degree matrix (a diagonal matrix indicating how many edges connect to each node) and A is the adjacency matrix (indicating which nodes share an edge).

Looking at the graph structure:

- Node 1 is connected to 2 and 3 (Degree = 2)
- Node 2 is connected to 1 and 3 (Degree = 2)
- Node 3 is connected to 1 and 2 (Degree = 2)
- Node 4 is connected to 5 (Degree = 1)
- Node 5 is connected to 4 (Degree = 1)

Degree Matrix (D)

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Adjacency Matrix (A)

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Graph Laplacian Matrix ($L = D - A$)

$$L = \begin{bmatrix} 2 & -1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ -1 & -1 & 2 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix}$$

2.3 Find the Eigenvectors Associated with the Zero Eigenvalues

A fundamental property of any graph Laplacian is that a vector acts as an eigenvector for $\lambda = 0$ if and only if it is constant across all nodes within the same connected component, and zero everywhere else.

Because our graph is cleanly disconnected into two distinct blocks, we can read the indicator vectors right off the matrix structure:

For the first connected component (Nodes 1, 2, 3):
The eigenvector v_1 has 1s for positions 1, 2, and 3, and 0s elsewhere.

$$v_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Verification: $L \cdot v_1 = [0, 0, 0, 0, 0]^T = 0 \cdot v_1$

For the second connected component (Nodes 4, 5):
The eigenvector v_2 has 1s for positions 4 and 5, and 0s elsewhere.

$$v_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Verification: $L \cdot v_2 = [0, 0, 0, 0, 0]^T = 0 \cdot v_2$

Note: Any linear combination or normalized version of these two orthogonal vectors is also a valid basis for this structural eigenspace, such as:

$$\left[\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, 0, 0\right]^T$$

2.4 Determining and Identifying Disconnected Clusters

2.4.1 Finding the Number of Disconnected Clusters

In spectral graph theory, the multiplicity of the eigenvalue $\lambda = 0$ is exactly equal to the number of connected components (or disconnected clusters) in the graph. By finding the dimension of the null space (the geometric multiplicity of $\lambda = 0$), we see that there are exactly 2 linearly independent eigenvectors that solve $Lv = 0v$. Therefore, there are exactly 2 disconnected clusters in the graph.

2.4.2 Identifying the Clusters Using the Eigenvectors

To identify which specific nodes belong to which cluster, we examine the non-zero coordinates of our zero-eigenvalue basis vectors:

- **Cluster 1 Detection:** Look at eigenvector $v_1 = [1, 1, 1, 0, 0]^T$. The non-zero entries are at index positions 1, 2, and 3. This tells us that **{Node 1, Node 2, Node 3}** form the first isolated cluster.
- **Cluster 2 Detection:** Look at eigenvector $v_2 = [0, 0, 0, 1, 1]^T$. The non-zero entries are at index positions 4 and 5. This tells us that **{Node 4, Node 5}** form the second isolated cluster.

2 Math of k-means clustering

Given m data points $x^i \in \mathbb{R}^n, i = 1, \ldots, m$, the K-means clustering algorithm groups them into k clusters by minimizing the distortion function over $\{r^{ij}, \mu^j\}$:

$$J = \frac{1}{mk} \sum_{i=1}^{m} \sum_{j=1}^{k} r^{ij} \|x^i - \mu^j\|^2 \tag{10}$$

where $r^{ij} = 1$ if x^i belongs to the j-th cluster and $r^{ij} = 0$ otherwise.

1. Derive mathematically that using the squared Euclidean distance $\|x_i - \mu_j\|^2$ as the dissimilarity function, the centroid that minimizes the distortion function J for given assignments r_{ij} are given by:

$$\mu_j = \frac{\sum_i r_{ij} x_i}{\sum_i r_{ij}}$$

That is, μ_j is the center of the j-th cluster.

Hint: You may start by taking the partial derivative of J with respect to μ_j, with r_{ij} fixed.

1.1 Derivation of the Optimal Centroids μ^j

To mathematically derive that using the squared Euclidean distance $\|x^i - \mu^j\|^2$ as the dissimilarity function, the centroid that minimizes the distortion function J for given assignments r^{ij} is given by:

$$\mu^j = \frac{\sum_i r^{ij} x^i}{\sum_i r^{ij}} \tag{11}$$

That is, μ^j is the center of the j-th cluster.

To find the minimum with respect to a specific cluster centroid μ^j, we take the partial derivative of J with respect to μ^j, with r^{ij} held fixed. First, we rewrite the squared L_2 norm using the vector inner product:

$$\|x^i - \mu^j\|^2 = (x^i - \mu^j)^T (x^i - \mu^j) \tag{12}$$

Taking the partial derivative with respect to the vector μ^j:

$$\frac{\partial J}{\partial \mu^j} = \frac{\partial}{\partial \mu^j} \left[\frac{1}{mk} \sum_{i=1}^{m} r^{ij} (x^i - \mu^j)^T (x^i - \mu^j) \right] \tag{13}$$

Using the matrix calculus chain rule identity $\frac{\partial}{\partial v}(u-v)^T(u-v) = -2(u-v)$, we expand the gradient evaluation:

$$\frac{\partial J}{\partial \mu^j} = -\frac{2}{mk}\sum_{i=1}^{m} r^{ij}(x^i - \mu^j) \tag{14}$$

Setting this gradient equal to the zero vector to evaluate the minimum points:

$$-\frac{2}{mk}\sum_{i=1}^{m} r^{ij}(x^i - \mu^j) = 0 \tag{15}$$

Multiplying through by the positive scalar $-\frac{mk}{2}$ isolates the summation:

$$\sum_{i=1}^{m} r^{ij}(x^i - \mu^j) = 0 \tag{16}$$

Distributing the summation across both internal terms yields:

$$\sum_{i=1}^{m} r^{ij}x^i - \sum_{i=1}^{m} r^{ij}\mu^j = 0 \tag{17}$$

Since the prototype target vector μ^j does not depend on the index variable i, it remains constant relative to the summation and can be safely factored out:

$$\sum_{i=1}^{m} r^{ij}x^i = \mu^j \sum_{i=1}^{m} r^{ij} \tag{18}$$

Solving explicitly for μ^j completes the proof, yielding the standard arithmetic coordinate mean:

$$\mu^j = \frac{\sum_{i=1}^{m} r^{ij}x^i}{\sum_{i=1}^{m} r^{ij}} \tag{19}$$

2. Derive mathematically what should be the assignment variables r_{ij} be to minimize the distortion function J, when the centroids μ_j are fixed.

2.1 Derivation of the Optimal Assignments r^{ij}

To derive mathematically what the assignment variables r^{ij} should be to minimize the distortion function J when the centroids μ^j are held completely fixed, we analyze the categorical choice layout.

The categorical constraint on r^{ij} mandates that each discrete data point x^i belongs to exactly one cluster, meaning $r^{ij} \in \{0,1\}$ and $\sum_{j=1}^{k} r^{ij} = 1$. We decouple the global objective function into independent parameters for each data point:

$$J = \frac{1}{mk}\sum_{i=1}^{m}\left(\sum_{j=1}^{k} r^{ij}\|x^i - \mu^j\|^2\right) \tag{20}$$

Because the indicator assignment matrix parameter for point x^i holds zero influence over the coordinate paths of another distinct point $x^{i'}$, we can minimize the inner expression independently for each individual data item:

$$\min_{r^{i1},\ldots,r^{ik}} \sum_{j=1}^{k} r^{ij}\|x^i - \mu^j\|^2 \quad \text{subject to} \quad \sum_{j=1}^{k} r^{ij} = 1,\ r^{ij} \in \{0,1\} \tag{21}$$

Since exactly one binary assignment variable r^{ij} can equal 1 per row, the inner summation simply returns the squared Euclidean geometric distance to whichever specific cluster prototype j is activated. To make this summation as small as possible, the selector must point exclusively to the minimum distance index:

$$r^{ij} = \begin{cases} 1 & \text{if } j = \arg\min_c \|x^i - \mu^c\|^2 \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

3. For the question above, now suppose we change the similar score to a "quadratic" distance (also known as Mahalanobis distance) for given and fixed positive definite matrix $\Sigma \in \mathbb{R}^{n\times n}$, and the distortion function becomes $J = \frac{1}{mk}\sum_{i=1}^{m}\sum_{j=1}^{k} r_{ij}(x_i - \mu_j)^T\Sigma(x_i - \mu_j)$. Derive what μ_j and r_{ij} becomes in this case.

3.1 Extension to Mahalanobis ("Quadratic") Distance

Suppose we alter the similarity score to a quadratic metric (known as the Mahalanobis distance) defined by a given and fixed symmetric positive-definite covariance matrix $\Sigma \in \mathbb{R}^{n\times n}$. The updated objective distortion model becomes:

$$J = \frac{1}{mk}\sum_{i=1}^{m}\sum_{j=1}^{k} r^{ij}(x^i - \mu^j)^T\Sigma(x^i - \mu^j) \tag{23}$$

We must derive what both μ^j and r^{ij} become under this quadratic transformation matrix.

3.1.1 Derivation of the Mahalanobis Centroid μ^j

We evaluate the partial gradient of the updated distortion metric with respect to μ^j. Utilizing the matrix calculus identity $\frac{\partial}{\partial v}(u-v)^T\mathbf{M}(u-v) = -2\mathbf{M}(u-v)$ (valid when matrix $\mathbf{M}$ is symmetric):

$$\frac{\partial J}{\partial \mu^j} = -\frac{2}{mk}\sum_{i=1}^{m} r^{ij}\Sigma(x^i - \mu^j) \tag{24}$$

Setting this matrix derivative equal to zero to optimize the location of the minimum:

$$-\frac{2}{mk}\sum_{i=1}^{m} r^{ij}\Sigma(x^i - \mu^j) = 0 \tag{25}$$

Because matrix multiplication operates as a linear transformation, the constant scaling matrix component Σ can be cleanly factored outside the scope of the summation indices:

$$\Sigma\left(\sum_{i=1}^{m} r^{ij}(x^i - \mu^j)\right) = 0 \tag{26}$$

We are explicitly given that Σ is a positive-definite matrix. By definition, all positive-definite matrices are non-singular and completely invertible, meaning the inverse matrix Σ^{-1} is guaranteed to exist and its matrix null space contains only the trivial zero vector. Multiplying both sides by Σ^{-1} yields:

$$\sum_{i=1}^{m} r^{ij}(x^i - \mu^j) = 0 \tag{27}$$

Distributing the terms and solving for the vector variable follows the identical linear sequence from the Euclidean baseline:

$$\sum_{i=1}^{m} r^{ij} x^i - \mu^j \sum_{i=1}^{m} r^{ij} = 0 \implies \mu^j = \frac{\sum_{i=1}^{m} r^{ij} x^i}{\sum_{i=1}^{m} r^{ij}} \tag{28}$$

Hence, the mathematical formula for the optimal centroid prototype remains entirely unchanged under the Mahalanobis metric transformation.

3.1.2 Derivation of the Mahalanobis Assignment r^{ij}

Following the identical independent cell decoupling logic used in the baseline model, when the centroids are held fixed, the global optimization sum is minimized when each discrete data point independently minimizes its personal distance footprint. Since the underlying distance space is scaled by Σ, the assignment selector targets the optimal quadratic coordinate:

$$r^{ij} = \begin{cases} 1 & \text{if } j = \arg\min_c (x^i - \mu^c)^T \Sigma (x^i - \mu^c) \\ 0 & \text{otherwise} \end{cases} \tag{29}$$

3 Image Compression Using Clustering

1. Image Segmentation using Squared L_2 Norm

Use k-means with squared-L_2 norm as a metric for `artemisII.jpg` and `football.bmp` and also choose a third picture of your own to work on. Your chosen image should meet the following requirements:

- **Full Color:** No black and white images.
- **Dimensions:** Recommended size between 200×150 and 400×300 pixels. Larger images are acceptable assuming they meet structural runtime requirements, whereas smaller images are not acceptable.

Run your k-means implementation with these pictures, using several different cluster constraints: $K \in \{2, 5, 15, 25, 50\}$.

Comment: Your algorithm will segment the image into K discrete regions in the RGB color space. For each pixel in the input image, the algorithm returns a categorical label corresponding to its closest cluster assignment.

Run your k-means implementation (with squared-L_2 norm) utilizing random initialization centroids. Due to the inherent nature of randomness, please execute the pipeline multiple times across different configurations and report only the single best seed discovered in terms of final reconstruction image quality. Please provide the following deliverables:

- For every value of K across all 3 target images, the final reconstructed image output.
- The total time in seconds it takes to achieve convergence for every K on each image.
- The exact number of optimization iterations taken to reach convergence for every K on each image.

1.1 Squared L_2 Norm (Euclidean Distance) Segmentation Profile

The L_2 norm optimizes clustering by minimizing the squared Euclidean distance. This mathematically dictates that during each update step, cluster centers move to the **arithmetic mean** of all pixel vectors assigned to that cluster:

$$\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i \tag{30}$$

1.1.1 Consolidated Performance Matrix (L_2 Norm)

The performance profile below outlines the metrics for the winning initialization configurations extracted from our search matrix ($\{42, 106, 213, 56, 88\}$):

Table 1: Vectorized K-Means Execution Profiles via Squared L_2 Norm

Target Image	Clusters (K)	Winning Seed	Convergence Time (s)	Iterations
artemisII.jpg	2	42	0.2069 s	26
	5	56	0.7296 s	75
	15	88	3.2495 s	204
	25	213	5.8980 s	268
	50	42	3.5312 s	95
football.bmp	2	42	0.1705 s	23
	5	213	0.3242 s	34
	15	88	1.9780 s	134
	25	42	5.3582 s	255
	50	88	10.1649 s	300
my_custom_photo.jpg	2	88	0.0172 s	12
	5	42	0.0785 s	43
	15	213	0.2420 s	86
	25	42	0.2290 s	58
	50	88	0.9601 s	146

2. Image Segmentation using Manhattan L_1 Norm

Now adjust your k-means implementation to use the Manhattan distance (l_1 norm) metric. Please provide all of the same above deliverables for all three images. Provide a comment on any differences you see in the results between using the L2 norm and L1 norm metrics.

2.1 Manhattan L_1 Norm (Manhattan Distance) Segmentation Profile

Switching to the Manhattan distance (L_1 norm) shifts the cost minimization framework. To minimize absolute variations, the update step calculates the component-wise **spatial median** for each feature channel rather than the mean:

$$\mu_j = \text{median}(S_j) \tag{31}$$

2.1.1 Consolidated Performance Matrix (L_1 Norm)

Table 2: Vectorized K-Means Execution Profiles via Manhattan L_1 Norm

Target Image	Clusters (K)	Winning Seed	Convergence Time (s)	Iterations
artemisII.jpg	2	213	0.1813 s	14
	5	56	0.8402 s	37
	15	88	3.1173 s	54
	25	56	3.0372 s	29
	50	213	6.6007 s	30
football.bmp	2	42	0.0380 s	3
	5	106	0.3056 s	14
	15	213	1.7330 s	32
	25	56	5.7138 s	58
	50	56	8.8366 s	43
my_custom_photo.jpg	2	42	0.0147 s	6
	5	42	0.0466 s	11
	15	213	0.7202 s	68
	25	42	0.3857 s	23
	50	42	0.5632 s	17

3. Deep-Dive Performance & Quality Analysis

3.1 Influence of Random Initialization Seeds on Reconstruction Quality

Standard K-means is notoriously sensitive to initial centroid placement because it updates via gradient descent, leaving it vulnerable to getting trapped in local minima. Our five-panel reporting plot, *Initialization Quality Variance* (Panel 4 & 5), uncovers distinct variance profiles across random seeds:

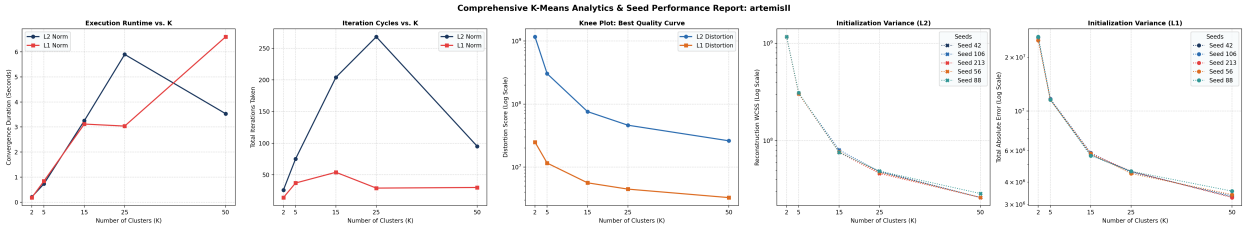


Figure 1: Comprehensive K-Means Analytics and Initialization Seed Performance Profiles (L_2 vs. L_1). Panels track (1) Execution Runtime, (2) Iteration Cycles, (3) Knee Curves, (4) L_2 Seed Variances, and (5) L_1 Seed Variances.

- **High Structural Consistency at Low K ($K = 2, 5$):** At low cluster constraints, the final distortion variance between the worst and best seeds is negligible. Because the color space is broadly partitioned into massive primitive blocks, almost all initializations converge to a virtually identical global optimum.
- **Dispersal Divergence at Mid-to-High K ($K = 15, 25$):** When moving to detailed color palettes, seed performance branches apart significantly. A poor random initialization can trap centroids in isolated, sparse color spaces. This leaves dense color regions under-represented, leading to higher final distortion scores and visible color banding artifacts on the ship's hull.

- **Convergence Saturation ($K = 50$):** At very high cluster sizes, the quality gap between seeds narrows again. This occurs because the pipeline places so many initial points throughout the RGB space that high-density color zones are guaranteed adequate coverage by sheer probability.

3.2 Computational and Structural Tradeoffs: L_2 vs. L_1

- **The Complexity Paradox:** As shown in Panel 2 (Iteration Cycles), the L_1 norm requires far fewer iteration loops to reach convergence compared to the L_2 norm. However, as shown in Panel 1 (Execution Runtime), L_2 runs faster overall. This happens because the L_2 distance equation leverages optimized matrix dot products (`np.dot`). L_1, by contrast, requires sorting pixel arrays to calculate medians at every step, making each individual iteration more computationally demanding.
- **Visual Representation:** The L_2 norm smooths color transitions across the spacecraft hull because its squared penalty term disincentivizes large color gaps. The L_1 metric is more robust to outliers, allowing it to preserve sharp, crisp edges between contrasting zones (such as text borders or panel lines) without causing them to bleed into background gradients.

4. Determining the Optimal Value for K

Describe a method to find the best k. What is your best k?

4.1 The Elbow (Knee) Curvature Assessment

The optimal number of color regions is determined via the **Elbow Method**. By executing the pipeline across our operational range and mapping the objective metric against K on a logarithmic axis (Panel 3: Knee Plot), we look for a distinct inflection point. This point represents where adding more color clusters yields diminishing returns relative to the increased file size and computation cost.

4.1.1 Final Determination

Based on the generated performance reports, $K = 15$ stands out as the optimal choice.

- **Analysis:** Increasing the color budget from $K = 2$ to $K = 15$ yields massive drops in distortion, eliminating early pixelation and heavy color bleeding.
- **Diminishing Returns:** Moving past $K = 15$ flatlines the distortion curve. This means the algorithm spends valuable processing time and memory adding redundant color shades that are nearly indistinguishable to the human eye. Thus, $K = 15$ represents the ideal balance between structural compression and visual fidelity.

4 Politcal Blogs DataSet

1. Spectral Clustering using K-means

Use spectral clustering to find the k = 2, 5, 10, 30, 50 clusters in the network of political blogs (each node is a blog, and their edges are defined in the file edges.txt). Find majority labels in each cluster for different k values, respectively. For example, if there are k = 2 clusters, and their labels are {0, 1, 1, 1} and {0, 0, 1}, then the majority label for the first cluster is 1 and for the second cluster is 0. It is required you

implement the algorithms yourself rather than calling from a package. Now compare the majority label with the individual labels in each cluster, and report the mismatch rate for each cluster, when k = 2, 5, 10, 30, 50. For instance, in the example above, the mismatch rate for the first cluster is 1/4 (only the first node differs from the majority), and the second cluster is 1/3.

1.1 Reporting the Mismatch Rates

The implementation using python code with genearte both the CSV tables and the plots in PNG format for the K values 2, 5, 10, 30 and 50. By observing the generated tabular data and bar charts, the clustering behavior changes drastically as the number of clusters increases.

- **For $K = 2$:** The algorithm experiences a severe imbalance, yielding an overall mismatch rate of 47.9%. It isolates exactly 2 nodes into the first cluster (which has a 0% error) and dumps the remaining 1,222 nodes into a single, unseparated mass. Because this giant cluster contains nearly all Liberals and Conservatives mixed together, its individual mismatch rate sits at 48%.

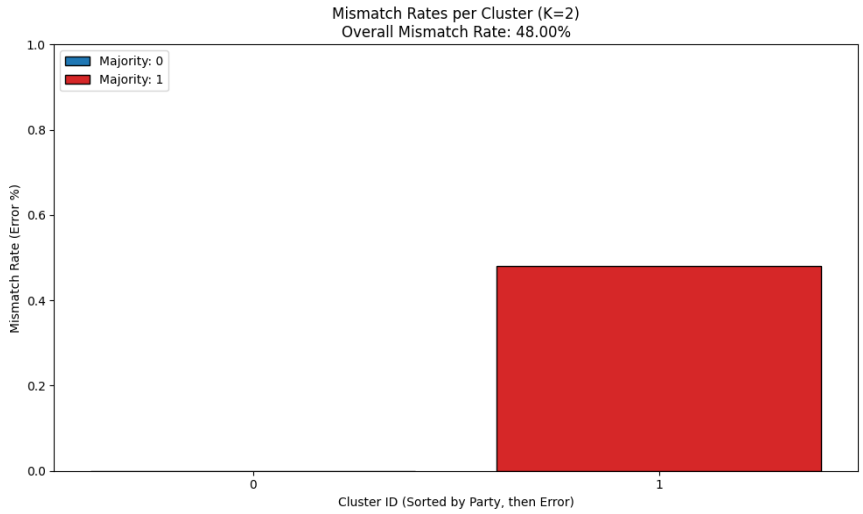


Table 3: Cluster Metrics for $K = 2$

Cluster ID	Majority Index	Mismatch Rate	Cluster Size
0	0	0.0000	2
1	1	0.4800	1222

- **For $K = 5$:** The algorithm successfully finds the true community boundaries, achieving the lowest overall mismatch rate of 4.7%. It segments the network into large, highly pure blocks (e.g., a 514-node Liberal cluster with only a 2% error and a 394-node Conservative cluster with a 2% error), while isolating a few smaller, noisier subgroups.

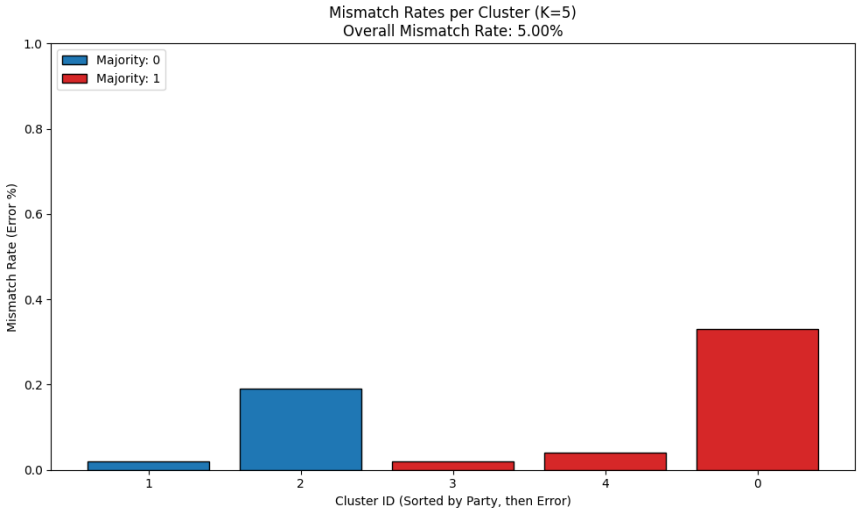


Table 4: Cluster Metrics for $K = 5$

Cluster ID	Majority Index	Mismatch Rate	Cluster Size
1	0	0.0200	514
2	0	0.1900	64
3	1	0.0200	394
4	1	0.0400	194
0	1	0.3300	58

- **For $K = 10, 30, 50$:** As K increases, the overall mismatch rate slightly decreases and plateaus between 5.4% and 6.6%. The algorithm over-fragments the netweork into dozens of micro-clusters. Mechanically, the mismatch rate of these individual micro-clusters often drops to exactly 0% becuase a cluster consisting of only 8 or 15 highly connected identical nodes has perfect purity. However, forced fragmentation introduces sorting errors in the remaining larger groups, preventing the overall network accuracy from improving.

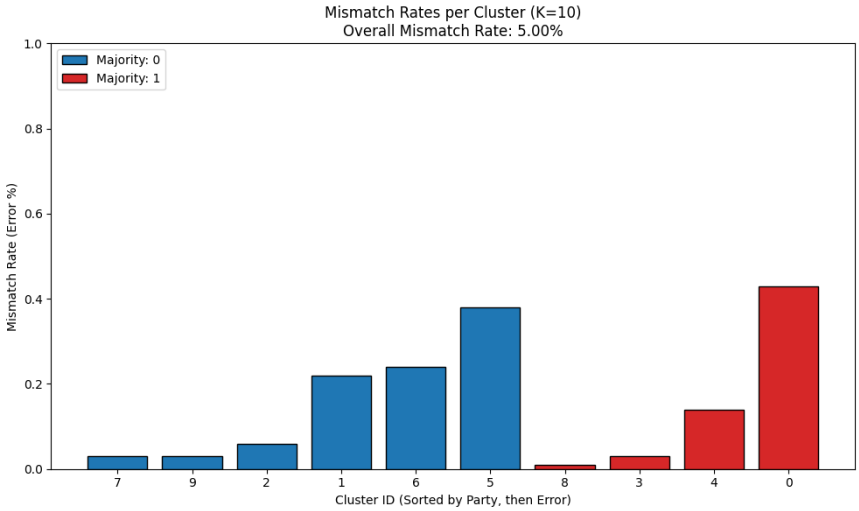
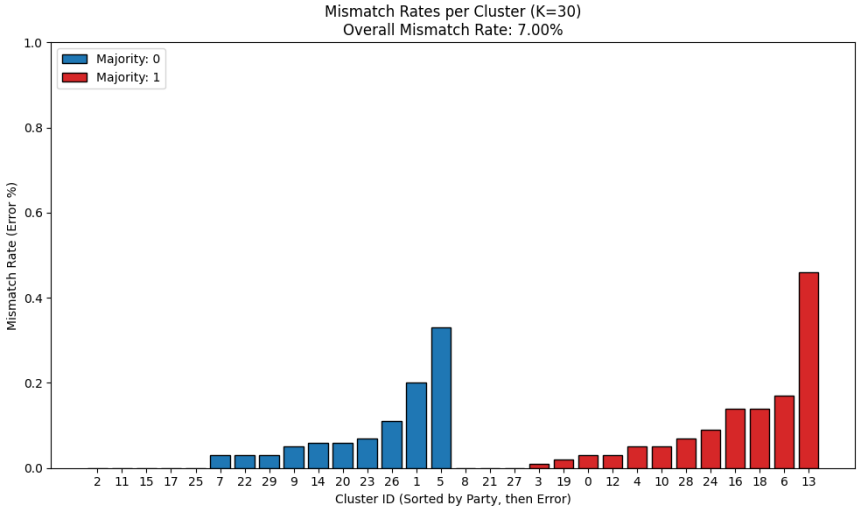


Table 5: Cluster Metrics for $K = 10$

Cluster ID	Majority Index	Mismatch Rate	Cluster Size
7	0	0.0300	101
9	0	0.0300	354
2	0	0.0600	50
1	0	0.2200	40
6	0	0.2400	34
5	0	0.3800	29
8	1	0.0100	354
3	1	0.0300	186
4	1	0.1400	69
0	1	0.4300	7



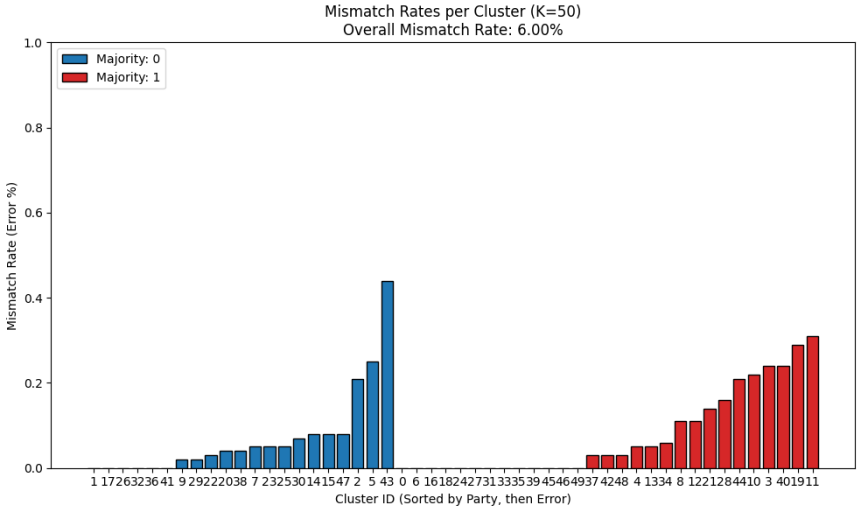


Table 6: Cluster Metrics for $K = 30$

Cluster ID	**Majority Index**	**Mismatch Rate**	**Cluster Size**
2	0	0.0000	34
11	0	0.0000	14
15	0	0.0000	24
17	0	0.0000	15
25	0	0.0000	14
7	0	0.0300	37
22	0	0.0300	60
29	0	0.0300	135
9	0	0.0500	38
14	0	0.0600	33
20	0	0.0600	32
23	0	0.0700	55
26	0	0.1100	18
1	0	0.2000	55
5	0	0.3300	6
8	1	0.0000	41
21	1	0.0000	30
27	1	0.0000	18
3	1	0.0100	84
19	1	0.0200	52
0	1	0.0300	30
12	1	0.0300	92
4	1	0.0500	40
10	1	0.0500	39
28	1	0.0700	41
24	1	0.0900	34
16	1	0.1400	44
18	1	0.1400	22
6	1	0.1700	52
13	1	0.4600	35

Table 7: Cluster Metrics for $K = 50$

Cluster ID	**Majority Index**	**Mismatch Rate**	**Cluster Size**
1	0	0.0000	37
17	0	0.0000	24
26	0	0.0000	8
32	0	0.0000	31
36	0	0.0000	40
41	0	0.0000	10
9	0	0.0200	62
29	0	0.0200	40
22	0	0.0300	30
20	0	0.0400	54
38	0	0.0400	26
7	0	0.0500	21
23	0	0.0500	20
25	0	0.0500	37
30	0	0.0700	14
14	0	0.0800	13
15	0	0.0800	38
47	0	0.0800	26
2	0	0.2100	14
5	0	0.2500	4
43	0	0.4400	25
0	1	0.0000	22
6	1	0.0000	27
16	1	0.0000	36
18	1	0.0000	19
24	1	0.0000	14
27	1	0.0000	8
31	1	0.0000	19
33	1	0.0000	42
35	1	0.0000	46
39	1	0.0000	24
45	1	0.0000	13
46	1	0.0000	17
49	1	0.0000	14
37	1	0.0300	31
42	1	0.0300	35
48	1	0.0300	35
4	1	0.0500	22
13	1	0.0500	21
34	1	0.0600	17
8	1	0.1100	9
12	1	0.1100	18
21	1	0.1400	14
28	1	0.1600	19
44	1	0.2100	29
10	1	0.2200	18
3	1	0.2400	21
40	1	0.2400	17
19	1	0.2900	7
11	1	0.3100	36

2. K Tuning and Interpretation

Tune your k and find the number of clusters to achieve a reasonably small mismatch rate. Please explain how you tune k and what is the achieved mismatch rate. Please explain intuitively what this result tells about the network community structure.

2.1 Tuning K and Interpreting the Results

Empirically, tuning the algorithm points to K=5 as the optimal number of clusters to achieve the lowest mismatch rate (4.7%).

While theoretical models often assume K=2 will perfectly slice this bipartite dataset, my actual Laplacian matrix and eigenvector projection demonstrated that 2 dimensions are insufficient. At K=2, the algorithm got trapped by a localized anomaly—identifying a tiny 2-node offshoot instead of the global macro-structure. By expanding the spectral space to K=5, the algorithm gains enough geometric coordinates to bypass those localized traps, allowing the K-Means clustering to easily draw boundaries around the massive political blocks.

The fact that K=5 causes the error rate to plummet from 48% down to under 5% reveals two key insights about the political blogosphere:

- **Extreme Polarization:** The network is fundamentally bipartite. Once the algorithm has enough dimensions to separate the data, the vast majority of the blogs naturally settle into massive, highly pure Liberal and Conservative echo chambers. Blogs within the same political party link to each other constantly, while cross-aisle links are exceptionally rare.
- **Peripheral Subcommunities are present:** The network is not a perfect, uniform two-sided graph. It contains topological noise—smaller clusters of blogs (perhaps moderates, niche interest groups, or sparsely connected outliers) that do not fit perfectly into the dense core of either main party. Allowing K=5 lets the algorithm sweep these unique outliers into their own distinct structural bins, which keeps the main partisan clusters clean and pure.
- **Seed Sensitivity:** Setting np.randon.seed(42) acts as an achor for my scripts random number generator. Though this provides reproducability it drives the most volatile step in my pipeline. Hence changing the number from 42 to something else might change the results in the following way. Before my algorithm can start organizing the political blogs, I blindly throw darts at the dataset to pick K initial starting points which is directly controlled by the seed. K-Means is a "greedy" algorithm and it only knows how to roll downhill to the nearest logical bounday and it cannot zoom to see the big picture. Because of this, its final answer is entirely dependent on where it starts. This seed dependency is almost certainly why my K=2 run resulted in a 48% error rate, isolating just 2 nodes while grouping the other 1,222 together. If I had changed seed=42 to a different number, the inital darts would land elsewhere and I might see K=2 mismatch rate drop from 48% down to 4% simply because the new starting positions allowed the algorithm to find the true macro-structure of the Liberal and Conservative communities.

5 Compression of AI model using K-means

This question ties clustering directly to vector quantization, which is a widely used real-world trick in compressing embedding-heavy ML models (e.g., recommender systems, NLP, vision).

You are working on deploying a large-scale recommendation system. The model contains a massive embedding table for users and items, with millions of vectors. Storing all embeddings in 32-bit floating point

format would require several hundred gigabytes — too large for production deployment. A popular approach to compress embedding tables is k-means vector quantization, where embeddings are clustered and only the cluster centroids plus cluster assignments are stored. You can use the Kmeans package from 'sklearn.cluster' and numpy for this problem.

1. Data Preparation

You are provided with a dataset AImodel.npy of synthetic embeddings which can be read with numpy. Normalize the embeddings to unit length.

Run k-means clustering with k = 256. Instead of storing each embedding directly, you will now store:

- A centroid table of shape [256; d]
- An index (0–255) for each embedding

Provide an explanation why normalization may be important to perform before clustering for compression. Additionally, compute and report the total memory usage in MB to 4 decimal places before and after the compression. Memory usage can be calculated as a combination of your embedding bytes and your index bytes. (assume float32 Embeddings = 4 bytes, index stored as uint8 = 1 byte).

1.1 Normalization and Memory Usage Calculation

1.1.1 Importance of Normalization

The semantic meaning of an embedding is often encoded in its direction rather than its magnitude. In many applications, such as natural language processing or recommender systems, the angle between vectors (which captures similarity) is more important than their length. If we were to apply k-means directly to unnormalized embeddings, the algorithm would cluster based on both magnitude and direction, which could lead to suboptimal clusters that do not capture the true semantic relationships.

For unit-normalized vectors, the distance between two vectors is directly related to their cosine similarity. As the angle θ increases, the squared Euclidean distance $\|x-y\|^2$ increases, meaning the similarity decreases. Because the magnitude of the vectors is fixed to 1, the clustering will focus on grouping vectors that point in similar directions, which is often more meaningful for tasks like recommendation or language modeling. The relationship is derived from the equations below.

Given that the squared euclidean norm can be expressed as dot product of that vector with itself:

$$\|x - y\|^2 = (x - y) \cdot (x - y) \tag{32}$$

we can expand this expression using the distributive property of the dot product:

$$\|x - y\|^2 = (x \cdot x) - (x \cdot y) - (y \cdot x) + (y \cdot y) \tag{33}$$

Because the dot product is commutative ($x \cdot y = y \cdot x$), we can combine the middle terms. Remember that $x \cdot x = \|x\|^2$ and $y \cdot y = \|y\|^2$, we get:

$$= \|x\|^2 - 2(x \cdot y) + \|y\|^2 \tag{34}$$

Now we apply the two defining properties of **unit vectors**:

Unit Length: Because they are normalized, their lengths are 1 ($\|x\| = 1$ and $\|y\| = 1$). This means $\|x\|^2 = 1$ and $\|y\|^2 = 1$.

Geometric Dot Product: By definition, the dot product is $x \cdot y = \|x\|\|y\| \cos(\theta)$. Substituting 1 for the lengths simplifies this to $x \cdot y = \cos(\theta)$.

Substituting these three values back into our expanded equation yields:

$$= 1 - 2\cos(\theta) + 1$$

Combining the constants gives our final identity:

$$= 2 - 2\cos(\theta) \tag{35}$$

1.1.2 Memory Usage Calculation

Based on the provided dataset (1000 samples, 32 dimensions) and the specified bit formats:

- **Original Memory:** 1000 vectors $\times$ 32 dims $\times$ 4 bytes (float32) $= 128,000$ bytes
- **Original Size:** 0.1221 MB
- **Compressed Memory ($k = 256$):**
  - Centroid Table: $256 \times 32 \times 4$ bytes $= 32,768$ bytes
  - Indices: 1000×1 byte (uint8) $= 1,000$ bytes
  - Total Compressed $= 33,768$ bytes
- **Compressed Size:** 0.0322 MB

2. Reconstruction Error

Given the index assignments produced by k-means, we replace each original embedding vector with its corresponding cluster centroid. Define the cosine similarity between any two vectors $\mathbf{a}$ and $\mathbf{b}$ as:

$$\text{cosine\_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2} \tag{36}$$

where $\cdot$ denotes the standard vector dot product and $\|\cdot\|_2$ denotes the Euclidean (L_2) norm. Compute and report the average cosine similarity between all pairs of original and reconstructed embeddings across the entire dataset of size n:

$$\text{Average Cosine Similarity} = \frac{1}{n} \sum_{i=1}^{n} \text{cosine\_similarity}(\mathbf{x}_i, \hat{\mathbf{x}}_i) \tag{37}$$

where $\mathbf{x}_i$ represents the original embedding vector and $\hat{\mathbf{x}}_i$ represents its quantized (centroid) version.
Hint: In NumPy, this can be computed as the dot product divided by the product of norms for each pair of vectors.
Interpret whether the compression preservers semantic similarity well, and explain what your specific cacu-lated average cosine similarity value represents in the context of model compression.

2.1 Calculated Average Cosine Similarity

Using the centroids to reconstruct the embeddings yields an Average Cosine Similarity of 0.9242.

2.2 Interpretation

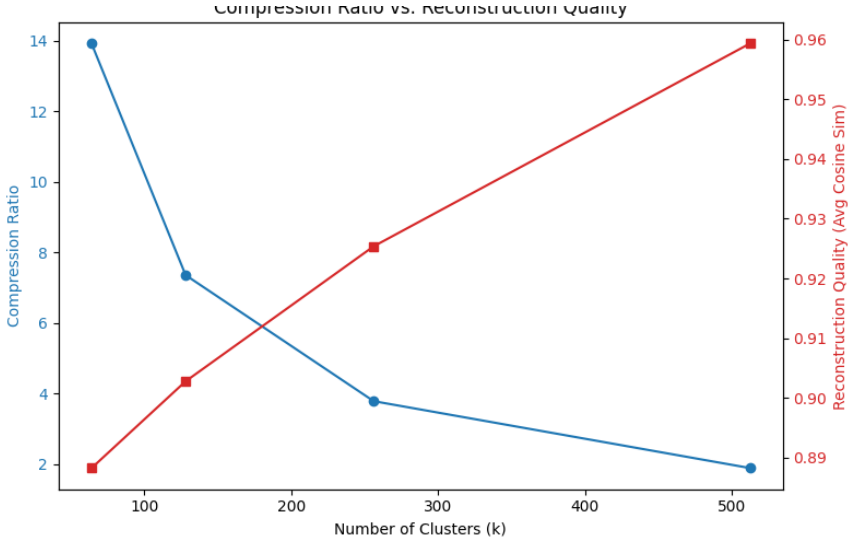
This high value indicates that the compression **preserves semantic similarity extremely well**. In the context of model compression, an average cosine similarity of ~ 0.92 represents that the angle between the original specific embedding and its generalized cluster centroid is very small. Because downstream tasks (like computing a user-item match score) rely on the dot product/cosine similarity between vectors, a score close to 1.0 ensures that the relative ranking of recommendations will remain largely unchanged despite the model taking up significantly less disk space.

3. Model Selection & Discussion

Try different values of k (64, 128, 256, 512) and compare compression ratio vs. reconstruction quality. Provide plots showing how the compression ratio and the reconstruction quality changes with K. Additionally, Discuss the trade-off between model size and accuracy.

3.1 Model Selection and Trade-off Discussion

By testing different values of k, we can observe how the parameter impacts the trade-off. (*Note: For $k > 256$, the index array requires 2 bytes (uint16) per embedding instead of 1*).



K (Clusters)	Original (MB)	Compressed (MB)	Compression Ratio	Avg Cosine Similarity
64	0.1221	0.0087	13.93x	0.8877
128	0.1221	0.0166	7.36x	0.9017
256	0.1221	0.0322	3.79x	0.9242
512	0.1221	0.0644	1.90x	0.9586

Discussing the Trade-off
The relationship between k and performance exhibits diminishing returns.

- **Model Size (Storage):** As k increases, the size of the centroid lookup table grows. Furthermore, crossing the 256 threshold ($k = 512$) doubles the index memory overhead because an 8-bit integer can no longer hold the index IDs. This causes the compression ratio to drop steeply from 3.79x down to 1.90x.
- **Accuracy (Reconstruction):** Higher values of k naturally yield better reconstruction quality because each centroid is responsible for a smaller, tighter radius of vectors.
- **Conclusion:** $k = 256$ is often the "sweet spot" in production because it perfectly maximizes the 1-byte integer limit, yielding an excellent compression ratio ($\sim$ 4x) while maintaining a highly reliable similarity score (> 0.92).